

Discrete Mathematics

Complete Exam Notes — Topics 1 to 3

Logic • Proofs • Set Theory

0 1

Contents

1 1 Logic and Its Applications

1.1 1Propositional Logic

1.1.1 1What is a Proposition?

Definition: Proposition

A **proposition** is a statement that is either **true** (T) or **false** (F), but *not both* at the same time.

Example: Identifying Propositions

- “ $2 + 2 = 4$ ” — **Yes**, it is a proposition (True).
- “It is raining today.” — **Yes**, it is a proposition (has a truth value).
- “What time is it?” — **No**, questions are not propositions.
- “ $x + 1 = 5$ ” — **No**, the truth depends on x (this is a *predicate*).

1.1.2 1Logical Connectives

Propositions are combined using **logical connectives**:

Name	Symbol	Read as	True when
Negation	$\neg p$	“not p ”	p is False
Conjunction	$p \wedge q$	“ p and q ”	both are True
Disjunction	$p \vee q$	“ p or q ”	at least one is True
Implication	$p \rightarrow q$	“if p then q ”	not ($p=T, q=F$)
Biconditional	$p \leftrightarrow q$	“ p iff q ”	both same truth value
Exclusive OR	$p \oplus q$	“ p xor q ”	exactly one is True

1.1.3 1Truth Tables for Each Connective

p	q	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$	$p \oplus q$
T	T	T	T	T	T	F
T	F	F	T	F	F	T
F	T	F	T	T	F	T
F	F	F	F	T	T	F

Note: The Implication Trap

$p \rightarrow q$ is **FALSE only** when p is True and q is False. In all other cases — including when p is False — it is **True**. This surprises many students. Think: “If you score 90, you get an A.” If you never score 90, the promise is never broken.

1.1.4 1Converse, Inverse, and Contrapositive

Given $p \rightarrow q$:

Name	Form	Logically equivalent to?
Original	$p \rightarrow q$	—
Converse	$q \rightarrow p$	Inverse
Inverse	$\neg p \rightarrow \neg q$	Converse
Contrapositive	$\neg q \rightarrow \neg p$	Original

Key Point

The **contrapositive** is logically equivalent to the original implication. This is very useful in proofs!

1.1.5 1Tautologies, Contradictions, and Contingencies

Definition: Tautology

- **Tautology:** Always True, regardless of truth values. Example: $p \vee \neg p$.
- **Contradiction:** Always False. Example: $p \wedge \neg p$.
- **Contingency:** Sometimes True, sometimes False. Example: $p \wedge q$.

1.1.6 1Logical Equivalences (Important Laws)

Two expressions P and Q are **logically equivalent** (written $P \equiv Q$) if their truth tables are identical.

Law	AND form	OR form
Identity	$p \wedge \mathbf{T} \equiv p$	$p \vee \mathbf{F} \equiv p$
Domination	$p \wedge \mathbf{F} \equiv \mathbf{F}$	$p \vee \mathbf{T} \equiv \mathbf{T}$
Idempotent	$p \wedge p \equiv p$	$p \vee p \equiv p$
Double Negation	$\neg(\neg p) \equiv p$	—
Commutative	$p \wedge q \equiv q \wedge p$	$p \vee q \equiv q \vee p$
Associative	$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$	(similar)
Distributive	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$	(similar)
De Morgan's Laws		
	$\neg(p \wedge q) \equiv \neg p \vee \neg q$	$\neg(p \vee q) \equiv \neg p \wedge \neg q$

Key Point

De Morgan's Laws are used everywhere — in proofs, programming, and circuit design. Memorise them! They say: “negate the whole $\Rightarrow$ flip $\wedge/\vee$ and negate each part.”

1.1.7 Logical Implication (Entailment)

Definition: Logical Implication

A set of premises **logically implies** conclusion C (written $P_1, P_2, \dots \models C$) if every assignment of truth values that makes all premises True also makes C True.

1.2 Predicate Logic and Quantifiers

1.2.1 Predicates

Definition: Predicate

A **predicate** is a statement that contains *variables* and becomes a proposition when those variables are assigned values. Write $P(x)$ to mean predicate P applied to variable x .

Example: Predicate

Let $P(x)$ mean “ $x > 5$ ”.

- $P(10)$ is True.
- $P(3)$ is False.
- $P(x)$ itself has no truth value until x is specified.

1.2.2 Quantifiers

Definition: Universal Quantifier $\forall$

$\forall x P(x)$ means “For **all** x , $P(x)$ is True.” It is False if even **one** value of x makes $P(x)$ False.

Definition: Existential Quantifier $\exists$

$\exists x P(x)$ means “There **exists** at least one x such that $P(x)$ is True.” It is True if **even one** value of x makes $P(x)$ True.

Example: Using Quantifiers

Let the domain be all integers.

- $\forall x (x^2 \geq 0)$ — **True** (squares are non-negative).
- $\exists x (x^2 = 4)$ — **True** (take $x = 2$ or $x = -2$).
- $\forall x (x > 0)$ — **False** ($x = -1$ is a counterexample).

1.2.3 Negating Quantifiers (De Morgan for Quantifiers)

Theorem: Negation of Quantifiers

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$

$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

Note: How to Remember

“Not everyone likes coffee” = “Someone does not like coffee.”

“No one passed” = “Everyone failed.”

1.2.4 Nested Quantifiers

When quantifiers appear one after another, **order matters!**

Example: Order of Quantifiers Matters

Domain = all real numbers.

- $\forall x \exists y (x + y = 0)$: For every x , we can find a y (namely $-x$). **True.**
- $\exists y \forall x (x + y = 0)$: There is one fixed y that works for all x . **False!**

Key Point

$\forall x \exists y$ and $\exists y \forall x$ have very different meanings. Always read left to right. Never swap quantifiers of different types without changing the meaning.

1.3 Computer Architecture Applications — Boolean Algebra

1.3.1 Connection between Logic and Circuits

In digital computers:

- **T** = 1 (high voltage) and **F** = 0 (low voltage).
- Logical connectives correspond to **logic gates**.

Gate	Symbol	Boolean expression
NOT	Bubble/triangle	$\bar{A}$ or $\neg A$
AND	D-shape	$A \cdot B$ or $A \wedge B$
OR	Curved shape	$A + B$ or $A \vee B$
NAND	AND + bubble	$\overline{A \cdot B}$
NOR	OR + bubble	$\overline{A + B}$
XOR	Curved + extra arc	$A \oplus B$

1.3.2 1Boolean Algebra Laws

Boolean algebra uses the same laws as propositional logic:

$$\overline{A + B} = \bar{A} \cdot \bar{B} \quad \overline{A \cdot B} = \bar{A} + \bar{B} \quad (\text{De Morgan's Laws})$$

1.3.3 1Functional Completeness

Definition: Functionally Complete Set

A set of connectives is **functionally complete** if every Boolean function can be expressed using only those connectives.

Theorem: Functionally Complete Sets

- $\{\neg, \wedge\}$ and $\{\neg, \vee\}$ are both functionally complete.
- $\{\text{NAND}\}$ alone is functionally complete: $\neg A = A \uparrow A$ and $A \wedge B = (A \uparrow B) \uparrow (A \uparrow B)$.
- $\{\text{NOR}\}$ alone is also functionally complete.

1.3.4 1Sum-of-Products (SOP) and Product-of-Sums (POS)

Definition: Sum-of-Products (Disjunctive Normal Form)

An expression is in **SOP** / **DNF** if it is a disjunction (OR) of conjunctions (ANDs):

$$(A \wedge \neg B) \vee (\neg A \wedge B \wedge C) \vee \dots$$

Each group is called a **minterm**.

Example: Building a Circuit from a Truth Table

Suppose output F is 1 only when $A = 1, B = 0$ and when $A = 0, B = 1$.

Minterms: $A\bar{B}$ and $\bar{A}B$.

SOP expression: $F = A\bar{B} + \bar{A}B = A \oplus B$ (XOR).

2 1

Proofs

2.1 1Definition of Proofs and Axiomatic Systems

Definition: Proof

A **proof** is a sequence of logically justified steps that starts from known facts (axioms or previously proved theorems) and arrives at a conclusion.

Definition: Axiomatic System

An **axiomatic system** consists of:

1. **Undefined terms** — primitive concepts accepted without definition.
2. **Axioms (postulates)** — statements accepted as true without proof.
3. **Defined terms** — concepts built from primitives.
4. **Theorems** — statements that are proved from axioms and definitions.

A good axiomatic system is *consistent* (no contradictions) and *complete* (every true statement can be proved) — though Gödel showed complete systems are impossible for sufficiently powerful mathematics.

Note: Commonly Used Axioms

- **Peano Axioms** — define the natural numbers.
- **ZFC Set Theory** — foundation of modern mathematics.
- **Field Axioms** — define real numbers.

2.2 1Rules of Inference

Rules of inference are **templates** for valid logical steps. If the premises are True, the conclusion must also be True.

Rule	Form	Plain English
Modus Ponens	$p, p \rightarrow q \vdash q$	“If if-then holds and p is true, so is q .”
Modus Tollens	$\neg q, p \rightarrow q \vdash \neg p$	“If q is false, so is p .”
Hypothetical Syll.	$p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$	“Chain of implications.”
Disjunctive Syll.	$p \vee q, \neg p \vdash q$	“Eliminate a false disjunct.”
Addition	$p \vdash p \vee q$	“OR anything in.”
Simplification	$p \wedge q \vdash p$	“Extract one conjunct.”
Conjunction	$p, q \vdash p \wedge q$	“Combine two truths.”
Resolution	$p \vee q, \neg p \vee r \vdash q \vee r$	Used in theorem provers.

Example: Modus Ponens in Action

Premises: “If it rains, the road is wet.” It is raining.

Conclusion: The road is wet.

Formally: Let $p = \text{“It rains”}$, $q = \text{“Road is wet”}$.

$p, p \rightarrow q \vdash q$. (Modus Ponens)

Example: Modus Tollens in Action

Premises: “If n is prime, $n \neq 1$.” We know $n = 1$.

Conclusion: n is not prime.

$\neg q, p \rightarrow q \vdash \neg p$. (Modus Tollens)

2.3 1Types of Proofs**2.3.1 1Direct Proof****Definition: Direct Proof**

To prove $p \rightarrow q$ **directly**: Assume p is True, then use definitions, axioms, and rules of inference to show q must also be True.

Example: Direct Proof

Theorem: If n is even, then n^2 is even.

Proof: Assume n is even. Then $n = 2k$ for some integer k .

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2).$$

Since $2k^2$ is an integer, $n^2 = 2 \cdot (\text{integer})$, so n^2 is even. $\square$

2.3.2 1Proof by Contrapositive**Definition: Proof by Contrapositive**

To prove $p \rightarrow q$: Instead, prove $\neg q \rightarrow \neg p$ (the contrapositive), which is logically equivalent.

Example: Proof by Contrapositive

Theorem: If n^2 is odd, then n is odd.

Contrapositive: If n is even, then n^2 is even. (Proved above!)

Since the contrapositive is true, the original statement is true. $\square$

2.3.3 1Proof by Contradiction

Definition: Proof by Contradiction (Reductio ad Absurdum)

To prove statement P : Assume $\neg P$ is True. Derive a **contradiction** (something that is always False). Since assuming $\neg P$ leads to a contradiction, P must be True.

Example: Classic Contradiction Proof

Theorem: $\sqrt{2}$ is irrational.

Proof: Assume $\sqrt{2}$ is rational. Then $\sqrt{2} = \frac{p}{q}$ where $p, q \in \mathbb{Z}$, $q \neq 0$, and $\gcd(p, q) = 1$ (fraction in lowest terms).

Squaring: $2 = \frac{p^2}{q^2}$, so $p^2 = 2q^2$.

This means p^2 is even $\Rightarrow p$ is even $\Rightarrow p = 2m$ for some m .

Then $(2m)^2 = 2q^2 \Rightarrow 4m^2 = 2q^2 \Rightarrow q^2 = 2m^2$.

So q^2 is even $\Rightarrow q$ is even.

But now both p and q are even, contradicting $\gcd(p, q) = 1$. **Contradiction!**

Therefore $\sqrt{2}$ is irrational. $\square$

2.3.4 1Proof by Cases (Exhaustive Proof)

Definition: Proof by Cases

Break the statement into **all possible cases** and prove each case separately. Every case must be covered — no gaps allowed.

Example: Proof by Cases

Theorem: For any integer n , $n^2 + n$ is even.

Case 1: n is even. Then $n = 2k$, so $n^2 + n = 4k^2 + 2k = 2(2k^2 + k)$. Even.

Case 2: n is odd. Then $n = 2k + 1$, so $n^2 + n = (2k + 1)^2 + (2k + 1) = 4k^2 + 4k + 1 + 2k + 1 = 4k^2 + 6k + 2 = 2(2k^2 + 3k + 1)$. Even.

In both cases, $n^2 + n$ is even. $\square$

2.3.5 1Proof by Mathematical Induction

Definition: Principle of Mathematical Induction

To prove $P(n)$ for all integers $n \geq n_0$:

1. **Base Case:** Prove $P(n_0)$ is True.
2. **Inductive Step:** Assume $P(k)$ is True for some arbitrary $k \geq n_0$ (the *inductive hypothesis*). Then prove $P(k + 1)$ is True.

Conclusion: $P(n)$ is True for all $n \geq n_0$.

Example: Induction: Sum Formula

Theorem: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ for all $n \geq 1$.

Base case ($n = 1$): LHS = 1; RHS = $\frac{1 \cdot 2}{2} = 1$. ✓

Inductive step: Assume $\sum_{i=1}^k i = \frac{k(k+1)}{2}$.

Show: $\sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}$.

LHS = $\sum_{i=1}^k i + (k+1) = \frac{k(k+1)}{2} + (k+1) = (k+1) \left(\frac{k}{2} + 1 \right) = \frac{(k+1)(k+2)}{2}$. ✓

By induction, the formula holds for all $n \geq 1$. □

2.3.6 1Strong Induction**Definition: Strong Induction**

Like regular induction, but the inductive hypothesis is stronger: assume $P(n_0), P(n_0 + 1), \dots, P(k)$ are **all** True, then prove $P(k + 1)$.

Example: Strong Induction: Fundamental Theorem of Arithmetic

Every integer $n \geq 2$ can be written as a product of prime numbers.

Base case ($n = 2$): 2 is prime. ✓

Inductive step: Assume every integer from 2 to k is a product of primes.

For $k + 1$: if it is prime, done. If not, $k + 1 = a \cdot b$ with $2 \leq a, b < k + 1$.

By the strong inductive hypothesis, both a and b are products of primes, so $k + 1$ is too. □

2.4 1Application in Program Correctness and Loop Invariants**2.4.1 1What is Program Correctness?****Definition: Partial and Total Correctness**

- **Partial Correctness:** If the program terminates, its output satisfies the postcondition (assuming precondition was satisfied).
- **Total Correctness:** Partial correctness *and* the program always terminates.

2.4.2 Hoare Triples

Definition: Hoare Triple

A **Hoare triple** is written $\{P\} S \{Q\}$ where:

- P is the **precondition** — what is true before executing S .
- S is the **program segment** (code).
- Q is the **postcondition** — what must be true after executing S .

It means: if P is true before S runs, then Q is true after S finishes.

2.4.3 Loop Invariants

Definition: Loop Invariant

A **loop invariant** is a condition (predicate) that is:

1. **True before the loop begins** (Initialisation).
2. **True after each iteration** (Maintenance) — if it was true before the iteration.
3. **Used with the loop's termination condition** to prove the postcondition (Termination).

Example: Loop Invariant for a Simple Loop

Goal: Prove that the following code computes $\text{result} = n!$.

```
result = 1;
i = 1;
while (i <= n):
    result = result * i;
    i = i + 1;
```

Loop Invariant: $\text{result} = i!$ (just before checking the loop condition, after updating).

More precisely, at the start of each iteration (when we check $i \leq n$): $\text{result} = (i - 1)!$ (since i is about to be used as the multiplier).

Let us use the invariant: **At the start of the k -th iteration, $\text{result} = (k - 1)!$.**

- *Initialisation:* Before the loop, $\text{result} = 1 = 0!$ and $i = 1$. ✓
- *Maintenance:* If $\text{result} = (i - 1)!$, after the body: $\text{result} \leftarrow (i - 1)! \cdot i = i!$, then $i \leftarrow i + 1$. So $\text{result} = ((i_{\text{new}}) - 1)!$. ✓
- *Termination:* Loop ends when $i > n$, i.e. $i = n + 1$. Then $\text{result} = (n + 1 - 1)! = n!$. ✓

Key Point

Loop invariants are the bridge between program code and mathematical proof. In an exam: state the invariant clearly, prove all three parts (Init, Maintenance, Termination).

3 1 Set Theory

3.1 1Russell's Paradox

3.1.1 1Naive Set Theory

In the early days, mathematicians defined sets very loosely: “*A set is any collection of objects satisfying some property P .*” — Cantor's naive definition.

The **Comprehension Axiom (Naive)**: For any property $P(x)$, the set $\{x \mid P(x)\}$ exists.

3.1.2 1Russell's Paradox

Definition: Russell's Paradox (1901)

Let $R = \{x \mid x \notin x\}$ — the set of all sets that do **not** contain themselves.

Ask: Is $R \in R$?

- If $R \in R$: then by definition, $R \notin R$. **Contradiction.**
- If $R \notin R$: then by definition, $R \in R$. **Contradiction.**

Either way we get a contradiction! This shows that *not every property defines a set*.

3.1.3 1Resolution

- **Type Theory** (Russell & Whitehead): Objects are arranged in a hierarchy of types; a set cannot contain itself because it would be at a lower type level than itself.
- **ZFC Set Theory** (most common today): Replaces the Comprehension Axiom with the *Separation Axiom*: sets can only be formed from *subsets* of already existing sets. There is no “set of all sets,” so the paradox never arises.

3.2 1Countability

3.2.1 1Set Cardinality and Bijections

Definition: Cardinality

The **cardinality** $|A|$ of a set A is the “size” of A — the number of elements. For infinite sets, cardinality is defined using **bijections** (one-to-one correspondences).

Definition: Same Cardinality

Two sets A and B have the **same cardinality** ($|A| = |B|$) if there exists a **bijection** $f : A \rightarrow B$ — a function that is both *injective* (one-to-one) and *surjective* (onto).

Property	Meaning
Injective (one-to-one)	$f(a_1) = f(a_2) \Rightarrow a_1 = a_2$ (no two inputs map to the same output)
Surjective (onto)	Every element of B is hit by some element of A
Bijjective	Both injective and surjective

3.2.2 1Countable Sets

Definition: Countably Infinite

A set A is **countably infinite** if there is a bijection $f : \mathbb{N} \rightarrow A$, i.e., its elements can be listed in a sequence: $a_1, a_2, a_3, \dots$

Definition: Countable

A set is **countable** if it is either *finite* or *countably infinite*.

Theorem: The Integers are Countable

$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ is countably infinite.

Proof: List them as $0, 1, -1, 2, -2, 3, -3, \dots$ — this is a bijection from $\mathbb{N}$ to $\mathbb{Z}$. $\square$

Theorem: The Rationals are Countable

$\mathbb{Q}$ is countably infinite.

Proof sketch: Arrange all fractions $\frac{p}{q}$ (with $q > 0$) in an infinite grid (rows by q , columns by p). Then **diagonalize** across the grid to produce a sequence that hits every fraction. $\square$

Theorem: Union of Countable Sets is Countable

If $A_1, A_2, A_3, \dots$ are all countable, then $\bigcup_{i=1}^{\infty} A_i$ is also countable.

3.3 1Uncountability

3.3.1 1The Real Numbers Are Uncountable

Theorem: Cantor's Diagonal Argument

$\mathbb{R}$ (and even the interval $(0, 1)$) is **uncountably infinite** — it cannot be put in bijection with $\mathbb{N}$.

Proof (by contradiction): Assume $(0, 1)$ is countable. Then list all its elements:

$$r_1 = 0.\mathbf{d}_{11}d_{12}d_{13}d_{14}\cdots$$

$$r_2 = 0.d_{21}\mathbf{d}_{22}d_{23}d_{24}\cdots$$

$$r_3 = 0.d_{31}d_{32}\mathbf{d}_{33}d_{34}\cdots$$

$$r_4 = 0.d_{41}d_{42}d_{43}\mathbf{d}_{44}\cdots$$

$$\vdots$$

Construct a new number $x = 0.x_1x_2x_3x_4\cdots$ where the digit x_n is chosen to *differ* from d_{nn} (e.g. if $d_{nn} \neq 5$, set $x_n = 5$; otherwise set $x_n = 6$).

Then x differs from r_n in the n -th decimal place for every n , so $x \neq r_n$ for all n .

But $x \in (0, 1)$, so it should be on the list. **Contradiction!**

Therefore $(0, 1)$ is uncountable, and so is $\mathbb{R}$. $\square$

Key Point

The key idea of Cantor's diagonal argument: Assume a complete list exists. Build a number that differs from every entry on the list at least in one digit. This number is not on the list — contradiction.

3.3.2 Cardinality Hierarchy

Set	Cardinality name	Symbol
Finite sets	Finite	$n \in \mathbb{N}$
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}$, all countably infinite sets	Aleph-naught	$\aleph_0$
$\mathbb{R}, (0, 1)$, all intervals	Cardinality of the continuum	$\mathfrak{c} = 2^{\aleph_0}$
Power set $\mathcal{P}(\mathbb{R})$	Strictly larger than $\mathfrak{c}$	$2^{\mathfrak{c}}$

Theorem: Cantor's Theorem

For any set A , the power set $\mathcal{P}(A)$ has strictly larger cardinality than A :

$$|A| < |\mathcal{P}(A)|$$

This means there is no largest infinity — there are infinitely many sizes of infinity!

3.4 Applications of Countability in Computability and Automata

3.4.1 Why Countability Matters in Computer Science

Theorem: The Number of Programs is Countable

Every computer program (in any language) is a finite string over a finite alphabet. The set of all finite strings over a finite alphabet is countable (enumerate them by length, then lexicographically). Therefore, the **set of all programs is countable**.

Theorem: The Number of Decision Problems is Uncountable

A **decision problem** is a function $f : \{0, 1\}^* \rightarrow \{0, 1\}$ (or equivalently, a subset of $\mathbb{N}$ or Σ^*). The set of all such functions has cardinality $2^{\aleph_0}$ — uncountable.

Theorem: Most Problems are Unsolvable

Since there are *countably many* programs but *uncountably many* decision problems, there must exist problems with **no program to solve them**. This is a cardinality argument — not a property of any specific problem.

3.4.2 The Halting Problem**Theorem: Halting Problem is Undecidable (Turing**

The Halting Problem: Given a program P and input I , does P halt on I ?

No algorithm can solve this for all (P, I) pairs.

Proof sketch (diagonal argument): Suppose a program H exists that decides halting. Build a program D :

- D takes a program X as input.
- D calls $H(X, X)$.
- If H says “ X halts on X ”, then D goes into an infinite loop.
- If H says “ X loops on X ”, then D halts.

Now ask: does D halt on input D ?

- If D halts on D : H would say so, causing D to loop. **Contradiction.**
- If D loops on D : H would say so, causing D to halt. **Contradiction.**

Therefore H cannot exist. The Halting Problem is undecidable. $\square$

3.4.3 Countability in Automata

Automaton / Language class	Count
Deterministic Finite Automata (DFA)	Countably many
Regular languages	Countably many (each described by a finite regex)
Context-Free Grammars / CFLs	Countably many
Turing Machines	Countably many
All languages over $\{0, 1\}^*$	Uncountably many

Key Point

Since the set of all Turing Machines is countable but the set of all languages is uncountable, there must be languages that no Turing Machine can recognise. This is the theoretical foundation of undecidability and incomputability.

3 1 Quick Revision Summary (Topics 1–3)

Topic 1: Logic

- A proposition is a statement with a definite truth value.
- Key connectives: $\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \oplus$.
- $p \rightarrow q$ is False **only** when $p = T, q = F$.
- Contrapositive $\equiv$ original; converse $\equiv$ inverse (but not the original).
- De Morgan: $\neg(p \wedge q) \equiv \neg p \vee \neg q$; $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
- Quantifiers: $\forall$ (all must work), $\exists$ (at least one works).
- Negate $\forall$ by using $\exists$ (and vice versa), then negate the predicate.
- NAND and NOR are functionally complete by themselves.

Topic 2: Proofs

- Proof types: Direct, Contrapositive, Contradiction, Cases, Induction.
- Induction: Base case + Inductive step (assume $P(k)$, prove $P(k+1)$).
- Strong induction: Assume all of $P(n_0), \dots, P(k)$ to prove $P(k+1)$.
- Loop invariant: True initially + maintained each iteration + gives postcondition at termination.
- Hoare triple $\{P\} S \{Q\}$: if P holds before S , then Q holds after.

Topic 3: Set Theory

- Russell's Paradox: $R = \{x \mid x \notin x\}$ leads to contradiction. Solved by ZFC.
- Two sets have same cardinality iff a bijection exists between them.
- $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$ are all countably infinite ($\aleph_0$).
- $\mathbb{R}$ is uncountable — proved by Cantor's diagonal argument.
- Programs are countable; decision problems are uncountable $\Rightarrow$ undecidable problems exist.
- The Halting Problem is undecidable (diagonal argument).
- Turing Machines are countable; languages are uncountable $\Rightarrow$ unrecognisable languages exist.